SAT Resetmaxxing for Fun and Profit

(Final Title TBD)

Martin S. Fixman, Frederik Rømming, and Sean B. Holden

University of Cambridge
United Kingdom
msf43@cl.cam.ac.uk

Abstract

We improve SATCoP performance by resetting the connection tableau often. This causes more and more varied SAT statements to be added to the SAT prover, which causes it to saturate faster. The resulting prover solves problems faster and in fewer steps than SATCoP and the current state-of-the-art tableau provers.

1 Introduction

2 Preliminaries

(TODO: Tableau or Tableaux? Bibliography is inconsistent.)

2.1 Tableau Solving Methods

Minimalist tableau-style provers based on Prolog have been used with great success since the release of leanTAP in 1995[1], and have consistently achieved high rankings in solving FOF and CNF problems in the Thousands of Problems for Theorem Provers (TPTP) library[11].

Rather than working like saturation provers such as Vampire[9], which repeatedly derive new clauses from the current clause set until they derive a contradiction or reach saturation, tableau-based provers attempt to create a Connection Tableau with ground terms. The prover builds a tree, where each node corresponds to a particular clause of the problem in conjunctive normal form, using one of the following four steps.

(TODO: Add a diagram and/or a description.)

Figure 1: Description of the operations of Tableau calculus[2].

leanCoP improves on earlier connection-tableau provers by combining a very small Prolog implementation with aggressive restricted backtracking[7, 6]. Successful reduction and extension steps commit with a Prolog *cut* (`!`), pruning alternative continuations and often reducing search substantially. To maximise the variety of substitutions in the tableau the maximum height of the tree is constrained, and each subsequent **Start** increases this height. A similar technique is later used in SATResetCoP, which is presented in this paper.

meanCoP revisits this design by separating the effects of reduction cuts and extension cuts, and by replacing leanCoP's particularly strong pruning with more selective exclusive

cuts[2]. In particular, meanCoP commits only when a successful proof step rules out sibling alternatives at the current choice point, and otherwise allows ordinary backtracking to continue[2]. This keeps much of the practical benefit of cut-based control while avoiding some of the unnecessary incompleteness and over-commitment of the original leanCoP strategy[2].

The leanCoP line has also been extended in several other directions. Functional and machine-learning-guided connection provers refine proof-step ordering and clause selection[4], rlCoP and plCoP add Monte Carlo Tree Search and reinforcement-learning-based control to leanCoP-style proof search[5, 12], and recent work on proof syntax and certification makes connection proofs easier to exchange and verify[3, 8].

2.2 SATCoP and Similar Methods

3 Research Vehicle

Talk about the Connections Prover[10].

4 Experimental Evaluation

5 Future Results

References

[1] Bernhard Beckert and Joachim Posegga. leanTAP: Lean tableau-based deduction. 15(3):339–358, 1995.

[2] Michael Färber. A Curiously Effective Backtracking Strategy for Connection Tableaux, 2024.

[3] Michael Färber, Cezary Kaliszyk, and Josef Urban. Machine Learning Guidance and Proof Certification for Connection Tableaux, 2018.

[4] Michael Färber, Cezary Kaliszyk, and Josef Urban. Machine Learning Guidance for Connection Tableaux. 65(2):287–320, 2021.

[5] Cezary Kaliszyk, Josef Urban, Henryk Michalewski, and Miroslav Olšák. Reinforcement learning of theorem proving. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31, pages 8836–8847. Curran Associates, Inc., 2018.

[6] Jens Otten. Restricting backtracking in connection calculi. 23(2–3):159–182, 2010.

[7] Jens Otten and Wolfgang Bibel. leanCoP: Lean connection-based theorem proving. 36(1–2):139–161, 2003.

[8] Jens Otten and Sean Holden. A Syntax for Connection Proofs, 2023.

[9] Alexandre Riazanov and Andrei Voronkov. The design and implementation of VAMPIRE. 15(2–3):91–110, 2002.

[10] Fredrik Rømming, Jens Otten, and Sean B. Holden. Connections: Markov Decision Processes for Classical, Intuitionistic and Modal Connection Calculi. In *Proceedings of the 1st International Workshop on Automated Reasoning with Connection Calculi*, volume 3613 of *CEUR Workshop Proceedings*, pages 107–118, 2023.

[11] Geoff Sutcliffe. The 12th IJCAR automated theorem proving system competition—CASC-J12. 38(1):3–20, 2025.

[12] Zsolt Zombori, Josef Urban, and Chad E. Brown. Prolog Technology Reinforcement Learning Prover: (System Description). In Nicolas Peltier and Viorica Sofronie-

Stokkermans, editors, *Automated Reasoning*, volume 12167, pages 489–507. Springer International Publishing, 2020.